PROJECT REPORT

Smart Agriculture Big Data Analytics with Scala and Spark

Crop stress prediction and live disease alerts on a Hadoop, Spark and Kafka pipeline

Subject: Big Data Analytics

Subject Code: 23AID302

Faculty In Charge: Dr. Sreeja B

Submitted by

Chaitanya Jayasurya J	CB.AI.U4AID24010
Sri Sai Nandini	CB.AI.U4AID24008
Sritha Manasa	CB.AI.U4AID24027
Siva Sashreek	CB.AI.U4AID24054

Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Coimbatore

October 2026

Contents

Contents 1
Abstract 2
1. Introduction 2
1.1 Problem Statement 2
1.2 Objectives 2
1.3 Why Big Data: the 5 V's 2
2. Literature Survey 3
3. Dataset 3
4. System Architecture 4
4.1 Cluster Topology 4
4.2 Data Flow 4
5. Methodology and Implementation 5
5.1 Sentinel-2 Preprocessing (Sentinel2Preprocess.scala) 5
5.2 ERA5 and SoilGrids Preprocessing 5
5.3 Canonical Join and MapReduce 5
5.4 Feature Engineering and Target 6
5.5 Machine Learning 6
5.6 Adaptive Kalman Filter on Spark 6
5.7 Real-Time Streaming 6
5.8 Leakage Audit 6
6. Spark Execution Model 7
7. Results 8
7.1 Stress Prediction 8
7.2 Other Results 8
8. Novelties 9
9. Challenges, Limitations and Future Work 10
9.1 Challenges 10
9.2 Limitations 10
9.3 Future Work 10
10. Conclusion 10
References 10

Abstract

This project builds an end-to-end big data pipeline that predicts crop stress and raises live crop-disease alerts for 19,916 farm fields near Bengaluru. Four data sources are combined: 47 GB of Sentinel-2 satellite imagery, ERA5 weather, SoilGrids soil maps and a live stream of IoT field sensors. All processing is written in Scala and runs on a Dockerised Hadoop cluster with one NameNode, five DataNodes (replication factor 3) and YARN.

Three Spark preprocessing jobs read the raw files from HDFS, apply 25 cleaning operations (cloud masking, NDVI computation, georeferencing, unit conversion, gap-filling) and write Parquet back to HDFS, producing a joined table of 393,340 field-by-date rows. A classic Hadoop MapReduce job computes monthly NDVI per tile. Spark MLlib models predict whether a field will be stressed at the next satellite pass, and are compared honestly with simple baselines. A Kafka and Spark Structured Streaming path filters live sensor readings with an adaptive Kalman filter and publishes disease and water-stress alerts. The live stream detected 58 of 59 at-risk simulated fields with no false alarms, and the adaptive Kalman filter cut sensor error by 54% when 15% of readings were faulty.

1. Introduction

1.1 Problem Statement

Crop stress and fungal disease are usually noticed only when damage is visible, which is too late to act. Early signals exist in satellite images, weather and soil properties, but they come in different formats and sizes and cannot be joined field by field on a single machine. Farms also need live monitoring, which requires processing a continuous stream of sensor readings.

1.2 Objectives

1. Store 47 GB of raw satellite, weather and soil data on a 5-DataNode Hadoop cluster.
2. Clean, georeference and join the data with Scala Spark jobs running on YARN.
3. Predict which 1.1 km fields will be stressed at the next satellite pass, with honest baselines.
4. Stream live IoT readings through Kafka and raise disease and water-stress alerts.
5. Prove that no future or ground-truth information leaks into the model.

1.3 Why Big Data: the 5 V's

- **Volume:** 47 GB of raw imagery, 141 GB stored in HDFS with three replicas.
- **Variety:** JPEG2000 imagery, NetCDF4 weather, GeoTIFF soil maps and JSON sensor messages.
- **Velocity:** 200 fields × 5 sensors, one reading per second each (1,000 readings per second).
- **Veracity:** clouds, duplicates and missing soil values must be detected and handled at scale.

- **Value:** turning this data into next-pass crop-stress predictions and live disease alerts for 19,916 fields, so farmers can act before yield is lost.

2. Literature Survey

Satellite vegetation indices are the standard way to monitor crop health at field scale. The Normalised Difference Vegetation Index, NDVI = (NIR − Red) / (NIR + Red), drops when plants are stressed, and Sentinel-2 provides it every five days at 10–20 m resolution [1]. ERA5 [2] and SoilGrids [3] supply the weather and soil context that explains why stress occurs.

Processing such data needs distributed systems. MapReduce [6] introduced fault-tolerant batch processing on commodity clusters; Spark [4] keeps data in memory and adds SQL, machine learning and streaming in one engine, with the Catalyst optimiser planning DataFrame queries [5]. Kafka [7] is the common way to move live sensor data into such systems. The Kalman filter [8] is the classic method for cleaning noisy sensor readings, and split conformal prediction [9] gives model outputs a guaranteed coverage level. This project combines these established techniques in one pipeline and adapts the Kalman filter so that its trust in each reading changes with the reading's observed reliability.

3. Dataset

Dataset	Source	Size / format	Used for
Sentinel-2 L2A	ESA Copernicus	41 zips, 47.0 GB, JPEG2000	Red, NIR and cloud mask → NDVI per field
ERA5	ECMWF Climate Data Store	NetCDF4, 29,565 daily rows	Weather: soil water, temperature, rain, radiation (2023)
SoilGrids	ISRIC	7 GeoTIFFs, 4.1 MB	pH, nitrogen, organic carbon, clay, sand, silt, water capacity
IoT sensors	Simulated, published to Kafka	JSON, 110,000 messages	Live disease and water-stress alerts
Final dataset	Our Scala ETL	393,340 rows, Parquet	19,916 fields × 40 dates; every model

Key attributes: tile, cell_row, cell_col, date, latitude, longitude, NDVI, NDVI standard deviation, cloud fraction, soil water (4 layers), temperature, dew point, wind, rain, solar radiation, pH, nitrogen, organic carbon, clay, sand, silt, water capacity, texture class.

The study area covers Sentinel-2 tiles T43PGQ and T43PHQ (about 110 km × 110 km each) from April 2023 to June 2024. One satellite file was a byte-identical duplicate and was removed by the pipeline. ERA5 was downloaded for 2023 only, which is recorded as a limitation in Section 9.

4. System Architecture

4.1 Cluster Topology

The cluster runs as 13 Docker containers on one laptop (32 cores, 16 GB given to Docker), defined in docker-compose.yml:

Component	Count	Role	Web UI
NameNode	1	HDFS master: file tree and block locations	localhost:9870
DataNode	5	Store 128 MB blocks; each block on 3 nodes	via NameNode
ResourceManager	1	YARN master: schedules Spark and MapReduce jobs	localhost:8088
NodeManager	2	Run executors and tasks (8 vcores, 4.5 GB each)	via YARN
Spark client	1	Submits jobs to YARN; Spark History Server	localhost:18080
Kafka + Kafka UI	2	Message broker (KRaft) and its web UI	localhost:8080
Timeline server	1	YARN job history	localhost:8188

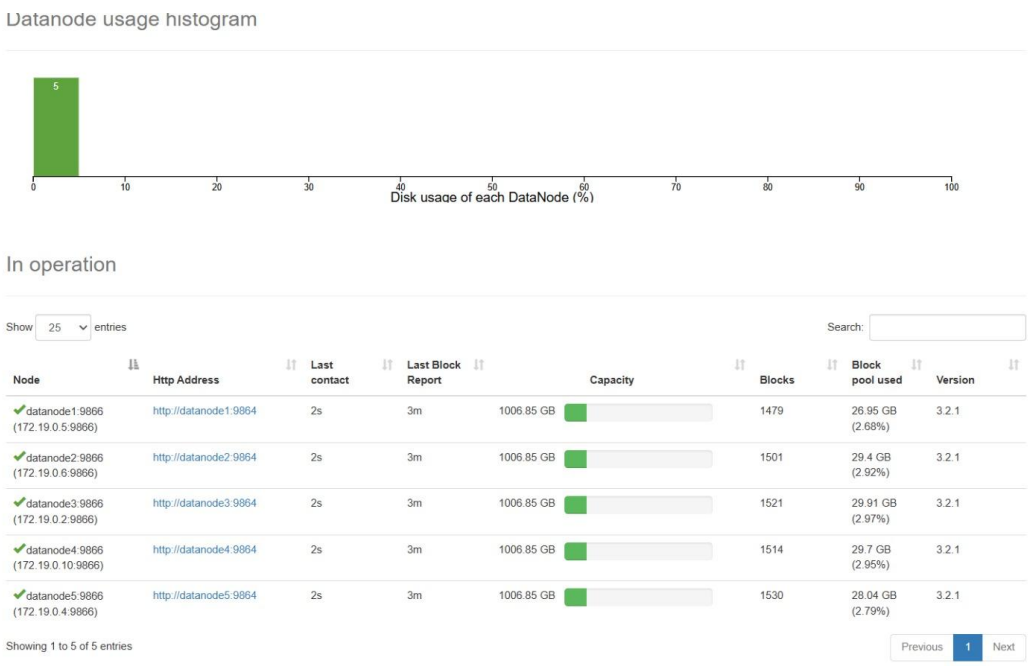


Figure 1. HDFS NameNode UI: five live DataNodes with blocks spread evenly by replication factor 3.

4.2 Data Flow

The system has three paths that share HDFS:

- **Batch:** raw files → HDFS raw zone → three Scala Spark preprocessing jobs → HDFS processed zone (Parquet).
- **Analytics:** canonical join → feature pipeline → Spark MLlib models → conformal prediction and decision engine; plus a Hadoop MapReduce job and a Kalman robustness experiment.

- **Streaming:** IoT simulator → Kafka topic iot_sensor_readings → Spark Structured Streaming (adaptive Kalman filter + risk rules) → Kafka topic crop_alerts and HDFS.

One command, bash scripts/run_pipeline.sh, runs all 14 stages in order and records a log of each stage.

5. Methodology and Implementation

All code is Scala 2.12 on Spark 3.5, built into one jar with sbt and submitted to YARN with spark-submit. The project has 67 Scala source files and 246 unit tests.

5.1 Sentinel-2 Preprocessing (Sentinel2Preprocess.scala)

Each satellite product is a 1.2 GB zip. One Spark task processes one product, so the 40 products are decoded in parallel. A custom HdfsSeekableChannel lets each task read only the three 20 m bands it needs directly from HDFS: 2.86 GB read out of 49.3 GB. Each pixel is cloud-masked with the Scene Classification Layer, converted to reflectance, turned into NDVI and averaged over a 100 × 100 grid of 1.1 km cells:

```
val zip = new ZipFile(new HdfsSeekableChannel(fs.open(path), size))
val red = Jp2Decoder.decode(entryBytes(zip, "_B04_20m.jp2"))
// ... same for B8A (near infrared) and SCL (scene classification)
if (SceneClass.isClearLand(scl)) {
  val ndvi = (nir - red) / (nir + red)
  sum(cell) += ndvi; clear(cell) += 1
}
```

Cell centres are converted from UTM to latitude/longitude in Scala (matching pyproj to seven decimal places), and cells with less than 30% clear land are dropped. Output: 393,340 cloud-free observations, partitioned by tile in HDFS.

5.2 ERA5 and SoilGrids Preprocessing

Era5Preprocess.scala decodes NetCDF4 with NetCDF-Java, applies scale and fill values, validates physical ranges, aggregates to daily values and derives relative humidity, vapour-pressure deficit and wind speed (29,565 rows, 0 missing). SoilGridsPreprocess.scala broadcasts the seven rasters to all executors, averages all soil pixels inside each 1.1 km cell, gap-fills empty cells from wider windows, converts units and derives the USDA texture class (19,916 profiles).

5.3 Canonical Join and MapReduce

CanonicalHistoricalDatasetBuilder.scala joins satellite, weather (nearest grid point, same date) and soil (same cell) into one table keyed by (tile, cell_row, cell_col, date), with validation checks. A classic Hadoop MapReduce job (NdviTileMonthDriver.scala) reads the exported NDVI and computes, per tile and month, the count, mean, minimum, maximum and standard deviation:

```
// Mapper (NdviTileMonthMapper): key = tile|year_month
outKey.set(NdviRecordParser.groupingKey(record.tile, record.yearMonth))
context.write(outKey, record.observation)
// Reducer (NdviTileMonthReducer): one pass over the key's values
while (iterator.hasNext) aggregator.accumulate(iterator.next())
val summary = aggregator.summary()
```

```
outValue.set((yearMonth +: summary.toTabDelimitedFields).mkString("\t"))
context.write(outKey, outValue)
```

5.4 Feature Engineering and Target

FeaturePipelineBuilder.scala builds leakage-safe features with Spark window functions: NDVI lag and change, rolling mean and standard deviation over the four previous observations, a baseline frozen strictly before the current date, soil properties, season and the current anomaly. The target is whether the field is stressed at the next satellite pass, meaning its next NDVI is more than one standard deviation below its own history. The data is split by date at 2024-04-07: 105,848 training rows and 48,747 test rows.

5.5 Machine Learning

TrainEvaluateJob.scala trains Spark MLlib pipelines (VectorAssembler followed by a classifier) with class weights: gradient-boosted trees, Random Forest and logistic regression, against a naive persistence rule and a majority-class baseline on identical rows. Hyper-parameters were chosen on a validation date inside the training period (2024-03-18); the test period was never used for any choice. Split conformal prediction ($\alpha = 0.10$) turns probabilities into prediction sets, and a decision engine maps them to LowRisk, Monitor or InvestigatePriority.

5.6 Adaptive Kalman Filter on Spark

Sensor and satellite readings are cleaned with a Kalman filter whose measurement noise adapts to each reading: $R_t = R_{base} / \max(\text{reliability}_t, \varepsilon)$. Reliability is computed only from what the sensor shows, using a physical-bounds check and the deviation from recent history. Because each Kalman step depends on the previous step's output, it cannot be a Spark window function; instead the data is repartitioned by field, sorted by date, and streamed through mapPartitions one field at a time:

```
obs.repartition($"tile", $"cell_row", $"cell_col")
  .sortWithinPartitions("tile", "cell_row", "cell_col", "date")
  .rdd.mapPartitions { rows =>
    cellByCell(rows).flatMap(runCellSequential)   // one field in memory
  }
```

5.7 Real-Time Streaming

IoTFieldSimulator.scala publishes readings from 200 fields × 5 sensors (soil moisture, soil temperature, air temperature, humidity, leaf wetness) to the Kafka topic iot_sensor_readings (6 partitions, key tile:row:col:sensor). CropHealthStreamingJob.scala reads the topic every 10 seconds, keeps a Kalman state per sensor across micro-batches with flatMapGroupsWithState, joins soil data and the model's satellite stress prediction, and applies agronomic rules (humidity ≥ 90%, wet leaves and 15–30 °C indicate fungal risk; soil moisture below half of field capacity indicates water stress). At-risk fields are published to the topic crop_alerts and saved to HDFS.

5.8 Leakage Audit

LeakageAudit.scala runs 23 automated checks: reflection on compiled classes (for example, the Kafka message type has no ground-truth field), scope-aware source scanning (future-target

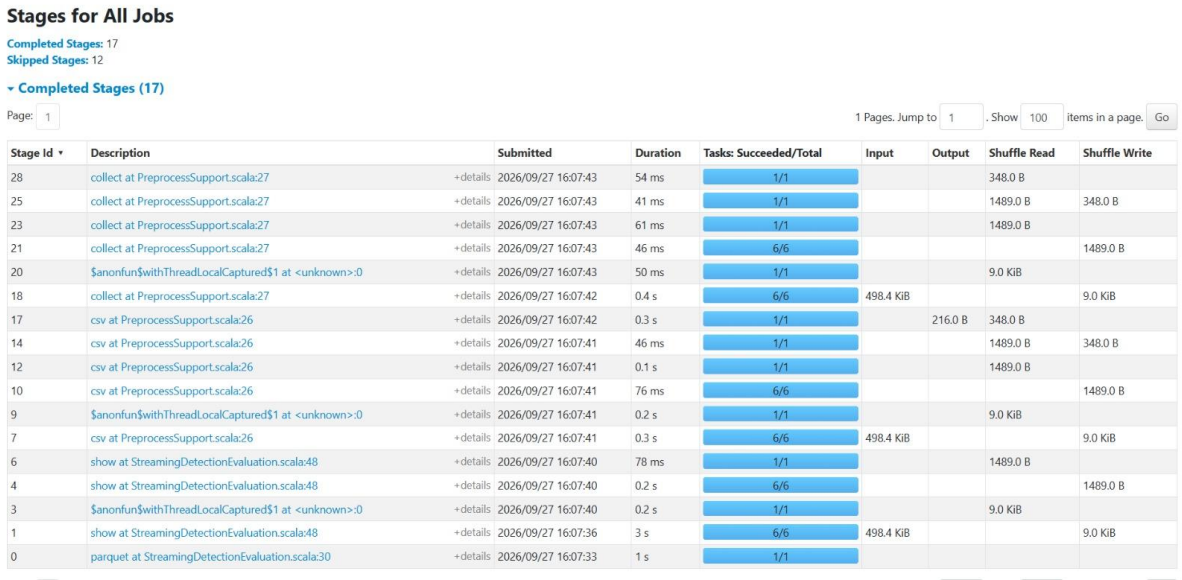
columns may appear only where the target is built), temporal checks on every window, training-only statistics, and five positive controls that plant violations and must catch them. All 23 checks pass.

6. Spark Execution Model

Spark records transformations lazily as a DAG; each action starts a job, the DAG scheduler cuts each job into stages at every shuffle, and each stage runs one task per partition. Measured from the Spark History Server for the full pipeline run:

Spark application	Jobs	Stages	Tasks	Max partitions
Sentinel2Preprocess	25	25	346	40
Era5Preprocess	38	38	587	64
SoilGridsPreprocess	23	23	161	16
CanonicalHistoricalDatasetBuilder	63	63	961	64
FeaturePipelineBuilder	121	141	3,468	64
KalmanRobustnessExperiment	53	56	989	64
CropStressTrainEvaluate	203	275	1,091	6
CropHealthStreaming	120	217	1,423	8
Total (10 applications)	674	866	≈ 9,100	–

- **Partitioning:** 40 partitions for satellite decoding (one per product); spark.sql.shuffle.partitions = 64 with adaptive query execution; HDFS output partitioned by tile, date and year.
- **Caching:** decoded products are persisted (MEMORY_AND_DISK) because two actions reuse them; the ERA5 grid and soil rasters are broadcast to every executor.
- **Executors:** 3 per job on the 2 YARN NodeManagers, 2 cores each, plus one driver and one ApplicationMaster.



Stages for All Jobs

Completed Stages: 17
Skipped Stages: 12

Completed Stages (17)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
28	collect at PreprocessSupport.scala:27 +details	2026/09/27 16:07:43	54 ms	1/1			348.0 B	
25	collect at PreprocessSupport.scala:27 +details	2026/09/27 16:07:43	41 ms	1/1			1489.0 B	348.0 B
23	collect at PreprocessSupport.scala:27 +details	2026/09/27 16:07:43	61 ms	1/1			1489.0 B	
21	collect at PreprocessSupport.scala:27 +details	2026/09/27 16:07:43	46 ms	6/6				1489.0 B
20	$anonfun$withThreadLocalCaptured$1 at <unknown>:0 +details	2026/09/27 16:07:43	50 ms	1/1			9.0 KiB	
18	collect at PreprocessSupport.scala:27 +details	2026/09/27 16:07:42	0.4 s	6/6	498.4 KiB			9.0 KiB
17	csv at PreprocessSupport.scala:26 +details	2026/09/27 16:07:42	0.3 s	1/1		216.0 B	348.0 B	
14	csv at PreprocessSupport.scala:26 +details	2026/09/27 16:07:41	46 ms	1/1			1489.0 B	348.0 B
12	csv at PreprocessSupport.scala:26 +details	2026/09/27 16:07:41	0.1 s	1/1			1489.0 B	
10	csv at PreprocessSupport.scala:26 +details	2026/09/27 16:07:41	76 ms	6/6				1489.0 B
9	$anonfun$withThreadLocalCaptured$1 at <unknown>:0 +details	2026/09/27 16:07:41	0.2 s	1/1			9.0 KiB	
7	csv at PreprocessSupport.scala:26 +details	2026/09/27 16:07:41	0.3 s	6/6	498.4 KiB			9.0 KiB
6	show at StreamingDetectionEvaluation.scala:48 +details	2026/09/27 16:07:40	78 ms	1/1			1489.0 B	
4	show at StreamingDetectionEvaluation.scala:48 +details	2026/09/27 16:07:40	0.2 s	6/6				1489.0 B
3	$anonfun$withThreadLocalCaptured$1 at <unknown>:0 +details	2026/09/27 16:07:40	0.2 s	1/1			9.0 KiB	
1	show at StreamingDetectionEvaluation.scala:48 +details	2026/09/27 16:07:36	3 s	6/6	498.4 KiB			9.0 KiB
0	parquet at StreamingDetectionEvaluation.scala:30 +details	2026/09/27 16:07:33	1 s	1/1				

Figure 2. Spark History Server: completed stages with tasks and shuffle read/write per stage.

7. Results

7.1 Stress Prediction

Test period April–June 2024, 48,747 rows:

Model	Accuracy	ROC-AUC	Balanced acc.	Recall (stressed)
Naive rule (stressed now → stressed next)	0.701	0.765	0.661	0.517
Random Forest + current anomaly	0.696	0.727	0.653	0.497
Logistic regression	0.639	0.707	0.500	0.001
Gradient boosting (primary, chosen on validation)	0.675	0.675	0.625	0.445
Random Forest, original features	0.629	0.656	0.530	0.174
Majority class	0.361	0.500	0.500	1.000

No learned model beat the naive persistence rule on the test months. Gradient boosting reached ROC-AUC 0.894 on the validation date but 0.675 on the test period, which shows strong seasonal drift between March and April–June. With only four training dates and no 2024 weather, the models could not learn that shift. The comparison against baselines was built in precisely so that this would be visible rather than hidden.

7.2 Other Results

Result	Value
Live streaming detection (simulated fields)	32 of 33 drying and 26 of 26 humid fields flagged; 0 false alarms in 141 normal fields
Adaptive Kalman, 15% outlier readings	54% lower error than a standard Kalman filter (2,000 series)

Result	Value
Adaptive Kalman, 20% biased readings	22% lower error
Adaptive Kalman, clean data	15% higher error (the trade-off of the method)
Conformal coverage (target 90%)	80.7%, reduced by the same seasonal drift
Decision engine	Fields flagged Investigate / Monitor / LowRisk were stressed 55% / 46% / 22% of the time
Leakage audit	23 / 23 checks pass



Figure 3. MapReduce result: mean NDVI per tile per month. Vegetation peaks in October 2023 and is lowest in the April 2024 dry season.

Insight: the previous pass's stress is the strongest early warning of the next, and live sensors add same-day disease warnings that satellites alone cannot give.

8. Novelties

1. **Adaptive, reliability-driven Kalman filter:** measurement noise changes per reading based only on observable signals; validated on 2,000 series × 11 corruption scenarios with an ablation of each component.
2. **Sequential Kalman filtering on Spark:** repartition by field, sort by date and stream through mapPartitions, so a fundamentally sequential algorithm runs distributed with bounded memory.
3. **A truly future target:** stress at the next satellite pass from information up to now, with a frozen baseline and a chronological split.

4. **Self-verifying leakage audit:** 23 automated checks, including positive controls that prove the detectors work.
5. **Honest multimodal integration:** satellite, weather, soil and IoT data in one pipeline, with every model compared against simple baselines on identical data.

9. Challenges, Limitations and Future Work

9.1 Challenges

- The pure-Java JPEG2000 decoder rejected Sentinel-2 files; we wrote our own JP2 box parser to extract the image codestream.
- Library version clashes between GeoTrellis and Spark on YARN were solved by shading libraries inside the jar.
- The Hadoop nodes run Java 8 while the build uses JDK 17; the code is compiled with -release 8.
- Running 5 DataNodes, YARN, Spark and Kafka within 16 GB required capping every daemon's memory.

9.2 Limitations

- ERA5 weather covers 2023 only, so the weather features are empty for the 2024 rows used by the models.
- SoilGrids and ERA5 stop at longitude 78°E, leaving part of tile T43PHQ without soil data.
- The IoT sensors are simulated, and all five DataNodes share one machine.

9.3 Future Work

- Connect real field sensors and run the cluster on separate machines.
- Add 2024 weather and more satellite dates, and retrain as seasons change.
- Merge small HDFS output files and add a farmer-facing alert application.

10. Conclusion

The project delivers a complete big data pipeline in Scala: 47 GB of raw data stored on a 5-DataNode HDFS cluster with replication 3, preprocessed by Spark jobs on YARN, aggregated by a MapReduce job, modelled with Spark MLlib and monitored live through Kafka and Spark Structured Streaming. All 14 pipeline stages run successfully from one command. The streaming system detects at-risk fields within one 10-second batch, the adaptive Kalman filter makes noisy sensors far more reliable, and the honest baseline comparison shows clearly where machine learning still needs more data.

References

1. ESA Copernicus. Sentinel-2 MSI Level-2A products. Copernicus Data Space Ecosystem.

2. Hersbach, H. et al. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 2020.
3. Poggio, L. et al. SoilGrids 2.0: producing soil information for the globe with quantified spatial uncertainty. SOIL, 2021.
4. Zaharia, M. et al. Apache Spark: a unified engine for big data processing. Communications of the ACM, 2016.
5. Armbrust, M. et al. Spark SQL: relational data processing in Spark. SIGMOD, 2015.
6. Dean, J., Ghemawat, S. MapReduce: simplified data processing on large clusters. OSDI, 2004.
7. Kreps, J., Narkhede, N., Rao, J. Kafka: a distributed messaging system for log processing. NetDB, 2011.
8. Welch, G., Bishop, G. An Introduction to the Kalman Filter. UNC Technical Report TR 95-041.
9. Angelopoulos, A. N., Bates, S. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. 2021.